It might seem that any key element with an associated count equal to or less than zero is not included in the Counter, but it is not actually true. The Counter continues to persist all key and counts irrespective of whether they are negative, positive, integer or float unless modified or deleted explicitly.

Let's take look at the following Counter:

```
>>> counter = Counter({'a': 1, 'b': 2, 'c': 3});
```

If you wish to modify the value, we can just assign a new value to the corresponding key.

```
>>> counter['a'] = 5
>>> counter
Counter({'a': 5, 'c': 3, 'b': 2})
```

If one wishes to delete the value from the Counter

```
>>> del counter['a']
>>> counter
Counter({'c': 3, 'b': 2})
```

It should be noted that a del operation in not equivalent to setting the count as 0.

- Another useful method of Counters and probably the most used one amongst all its methods is `most_common([n])`. The method name is self-explanatory, as it fetches the 'n' most common elements from the given Counter. Of any two elements in a Counter, an element is more common than the other when it has a higher number of occurrences.

```
>>> counter = Counter({'a':3,'b':3,'c':2,'d':
2,'e':1});
>>> counter.most_common(1)
[('a', 3)]
>>> counter.most_common(3)
[('a', 3), ('b', 3), ('c', 2)]
```

In the case of multiple elements having the same count, the most common element is picked arbitrarily.

- A counter is a pretty useful instrument when we want a quick count of uniquely identifiable repetitive objects. It supports convenient and rapid tallies of objects. Let us assume you have a list of characters and you wish to find out how many times each character appears in that list. Let's consider a list l with the following data:

```
>>> l = ['a','b','c','d','e','e',
'd','c','b','c','c','c']
```